

Week 11: Recursive Language Models & The Great Context Debate

The cutting edge of context processing, and the real-world debate that proves it matters.

WEEK

11

DATE

28 MAR 2026

FOCUS

RLMS & AGENTIC SEARCH

Last week we saw how Naive RAG catastrophically degrades on production data due to Context Rot. This week, we go beyond retrieval entirely. Recursive Language Models (RLMs) treat context as a programmatic environment the model explores with code — eliminating context rot at the architectural level. We then dissect the billion-dollar debate: Claude Code's agentic search vs. Cursor's indexed RAG, revealing that understanding **how** the model relates to its context is the single most important design decision in modern AI.

Quick Links & Resources

Resource	Link
 Lecture Slides	Google Drive
 Colab Notebook	Google Colab (Local Copy: Codes/class_8.ipynb)
 Excalidraw Board	Interactive Whiteboard

Table of Contents

#	Section	Core Concept
	The Core Thesis	Why stuffing the context window is a losing strategy.
1	The RLM Paradigm Shift	Context as a variable vs. context as direct input.
2	How RLM Works: Architecture	Root LLM, REPL, and the active exploration loop.
3	RLM Step-by-Step Example	Dynamic slicing and sub-agent synthesis on a 500-page report.
4	The Power of Recursion	Human-like research workflows at infinite depth.
5	Why RLM Beats Everything	No context rot, infinite scale, cost-efficient, active exploration.
6	Claude Code vs Cursor	Two billion-dollar products, two opposite architectures.
7	Agentic Search Deep Dive	The toolkit & philosophy behind Claude Code's shell-first approach.
8	Why Agentic Search Works	Precision, freshness, security — and the Anthropic quote.
9	The Indexed RAG Approach	Tree-sitter ASTs, Merkle sync, semantic search at scale.

#	Section	Core Concept
10	Comparing the Struggles	Honest trade-offs between both approaches.
111	The 2026 Consensus	The best teams use both — and the paradigm shift that matters.
	Hands-On: Notebook Walkthrough	Build a full RLM from scratch — REPL, agent loop, sub-LLMs.
	Excalidraw Masterclass	Contextual Retrieval, agent internals, and causal masking.
	Further Reading & Resources	Paper links, GitHub repos, blog posts.

The Core Thesis

"Just give the model more context! 1M+ token windows solve everything."

That belief is the root of every production failure we've studied.

Week 10 showed us *Context Rot* — how models catastrophically degrade as context length grows, even on trivial tasks. This week, we answer the follow-up question: **If stuffing the context window doesn't work, then what does?**

The Evolution of Context Management:

Week 09: Naive RAG

→ Stuff Top-K chunks into prompt → Works on demos, breaks on prod

Week 10: Advanced RAG

→ Hybrid Search, Reranking, HyDE → Better retrieval, same paradigm

Week 11: RLM

→ Context as environment → Model writes code to explore

Agentic Search

→ No index at all → Model uses shell tools live

[!CAUTION] **Don't blindly stuff the context window.** Every token you add without a surgical reason is a bet *against* the accuracy of the model's answer. The frontier is about giving the model *active control* over what it reads.

1 The RLM Paradigm Shift

Based on research by **Alex Zhang, Tim Kraska & Omar Khattab** at MIT CSAIL ([Paper: arxiv.org/abs/2512.24601](#)), Recursive Language Models completely rethink how LLMs interface with large inputs.

The Fundamental Difference

Traditional LLM	RLM Paradigm
-----------------	--------------

	Traditional LLM	RLM Paradigm
Context	The prompt IS the input to the network.	The prompt is stored as a VARIABLE .
Processing	Every token passes through the attention mechanism.	Model interacts with context via CODE .
Scaling	More tokens → attention is stretched thinner → accuracy drops.	More tokens → just recurse deeper → accuracy stays constant.
Result	Context rot happens. Inevitably.	No context rot. Infinite scale.

The Analogy

Think of the difference like this:

- **Traditional LLM:** You hand someone the *entire* Encyclopedia Britannica and say "*Find the population of Tokyo.*" They must read through every page, and their eyes glaze over after page 50.
- **RLM:** You hand someone the *index* of the Encyclopedia and a pair of scissors. They look up "Tokyo → Volume T, Page 847", cut out that specific page, and read only the relevant paragraph.

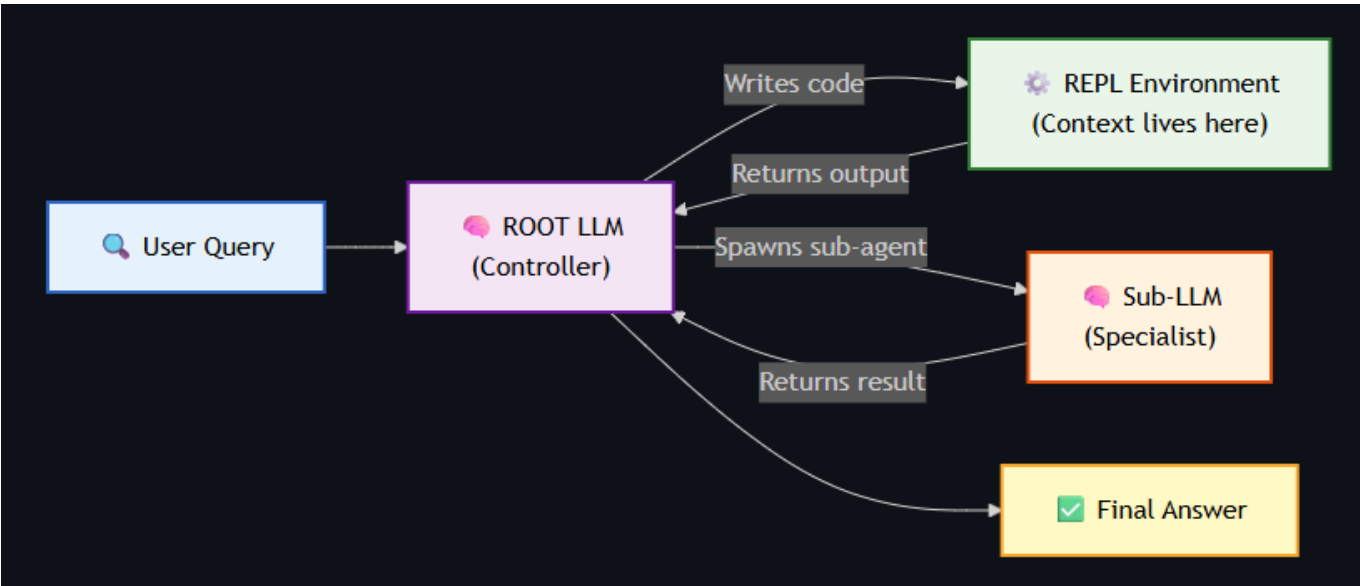
The second person never experiences cognitive overload. The RLM never experiences context rot.

[!IMPORTANT] **The key insight:** The model should NOT passively receive context. It should *actively explore* context using tools — just like a human researcher navigates a library.

2 How RLM Works: The Architecture

Instead of feeding the document into the prompt, an RLM places the document into a **runtime environment** and gives the model tools to explore it programmatically.

The Three Core Components



Component	Role	Critical Detail
-----------	------	-----------------

Component	Role	Critical Detail
 ROOT LLM	The Controller / Researcher	Receives only the query — never the raw document. Plans what to explore, writes Python code, orchestrates sub-agents.
 REPL Environment	The Runtime	Stores the massive input as a Python variable (<code>context</code>). Executes the Root LLM's code. Returns <code>print()</code> output back to the model.
 Sub-LLM	The Specialist	Called via <code>llm_query(question, chunk)</code> . Processes a <i>small</i> slice of context for tasks requiring <i>understanding</i> (not just counting). Its result returns as a string — it never enters the Root LLM's context window.

The Available Tools

Tool	Operation	Analogy
<code>context[:1000]</code>	Peek — Read the first N characters	Skimming the table of contents
<code>re.findall(...)</code>	Search — Find patterns via regex	Using Ctrl+F in a PDF
<code>context[a:b]</code>	Slice — Extract a specific range	Turning to a specific page
<code>llm_query(q, chunk)</code>	Recurse — Spawn a sub-LLM on a chunk	Asking a colleague to read a section for you

 The Agentic Loop

```
Write Code → Execute in REPL → See Output → Iterate → FINAL()
```

This is the exact same **ReAct pattern** from Week 08 (THOUGHT → ACTION → OBSERVATION), but the "tool" is the model's own context. Same architecture, different application.

3 RLM Step-by-Step Example

Let's trace a concrete scenario from the slides.

Query: "What was ACME Corp's Q3 APAC revenue?" **Context:** A 500-page annual report (millions of characters), stored as a Python variable.

Step	Name	What the Root LLM Does	Code Written
1	Initial Peek	Reads the first 2000 chars to discover the document's structure and Table of Contents.	<code>print(context[:2000])</code>
2	Targeted Search	Runs a regex to locate every mention of "APAC" or "Asia Pacific" and their character offsets.	<code>re.findall(r'APAC Asia Pacific', context)</code>

Step	Name	What the Root LLM Does	Code Written
3	Context Slicing	Precisely slices the section around the APAC mention and delegates it to a Sub-LLM for deep extraction.	<code>result = llm_query(q, context[45000:52000])</code>
4	Final Synthesis	Receives the Sub-LLM's concise answer and formats the final response for the user.	<code>FINAL(result)</code>

[!TIP] **The root model never processed the full 500 pages.** It peeked at the structure (~2K chars), ran a \$0 regex search, sliced ~7K chars for a sub-LLM, and synthesized. Total tokens through transformer attention: < 10K. Total document size: potentially 10M+. **Zero context rot.**

4 The Power of Recursion

RLM mimics how humans actually conduct research — we never read an entire library; we navigate hierarchically.

Human-like Research Workflow

Step	Human Researcher	RLM Equivalent
1. Scan	Check Table of Contents / Index	<code>context[:2000]</code> — peek at structure
2. Narrow	Identify relevant chapters	<code>re.findall(...)</code> — locate target sections
3. Deep-dive	Read specific sections carefully	<code>rlm_agent(q, chunk)</code> — sub-LLM reads closely
4. Synthesize	Combine findings into an answer	<code>FINAL(combined_results)</code>

The Recursive Tree

When a sub-agent encounters a chunk that's *still* too large, it can spawn its own sub-agents — going arbitrarily deep:

```
Root LLM (Depth 0) - "What was ACME's Q3 APAC revenue?"
├─ rlm_agent(query, chunk_1) [Depth 1] - Financial Sections
│   └─ rlm_agent(q, sub_1a) [Depth 2] - Q3 Breakdown
│       └─ rlm_agent(q, sub_1b) [Depth 2] - APAC Regional Data
├─ rlm_agent(query, chunk_2) [Depth 1] - Executive Summary
│   └─ rlm_agent(q, sub_2a) [Depth 2] - Revenue Highlights
```

Scalability: 10M tokens? 100M tokens? No problem. Each recursive node sees only a *small, highly focused* piece of text. The findings bubble back up the chain. You never hit a ceiling — you just add another layer of recursion.

5 Why RLM Beats Everything

Advantage	Explanation	Why It Matters
-----------	-------------	----------------

Advantage	Explanation	Why It Matters
1. NO CONTEXT ROT	Each LLM call sees only a small, focused piece . The "lost in the middle" problem is architecturally eliminated.	The #1 failure mode of production RAG — gone.
2. INFINITE SCALE	10K tokens → 100M tokens. The approach stays identical — just recurse deeper.	No need to wait for 10M token context windows.
3. ACTIVE EXPLORATION	The model isn't a passive sponge. It chooses its path , uses Python to filter noise, and only reads what matters.	Dramatically better precision than Top-K retrieval.
4. COST-EFFECTIVE	Irrelevant text is filtered by string operations (\$0 compute), not by the transformer (\$\$\$). Only "gold" tokens hit the GPU.	Orders of magnitude cheaper at scale.

 Mind-Blowing Empirical Results

Model Configuration	vs. Baseline	Performance Gain	Context Capacity
RLM (GPT-5-mini)	vs Vanilla GPT-5	+34.2 Points	100x Beyond Context Window
RLM (Qwen3-8B)	vs Base Qwen3-8B	+28.3% Average	Infinite Recursion
RLM (Llama-3-70B)	vs Vanilla Llama-3	+19.5 Points	Scales to 10M+ Tokens

"A mini model using RLM beats a full model with raw context." Handles inputs up to **100x beyond the model's context window** at comparable or lower cost.

6 Claude Code vs Cursor: The Great Code Search Debate

How do billion-dollar AI coding products solve the problem of navigating massive codebases? The answer reveals two fundamentally opposite philosophies about how models should relate to context.

Feature	Claude Code	Cursor
Architecture	Agentic Search	Indexed RAG
Core Method	Shell tools (grep , glob , ls , cat)	Embeddings + Vector databases
Index Required?	✗ None. Reads live from disk.	☑ Background indexing pipeline.
Philosophy	Active exploration over passive consumption	Smart pre-computation over real-time exploration
Best For	Deep, autonomous, multi-file refactors	Quick semantic discovery, rapid UI iteration

Both are billion-dollar products solving the same problem differently. Understanding *why* each approach fits its domain is what separates someone who *uses* AI tools from someone who *builds* them.

7 Agentic Search Deep Dive (Claude Code)

Claude Code's approach is radically simple: **no embeddings, no index, no vector database**. The model actively explores the filesystem using basic shell tools.

The Minimal Toolkit

Tool	Purpose
<code>ls</code>	List directory contents
<code>glob</code>	Pattern-match file paths
<code>grep</code>	Search for text patterns across files
<code>cat</code>	Read entire file contents

No Embeddings. No Index.

The Agentic Process

The model doesn't search — it *investigates*. Each step informs the next, like a detective following leads:

Step	Question the Model Asks	Tool Used
1. Discovery	"What files exist here?"	<code>ls</code> , <code>glob</code>
2. Navigation	"Where is this function defined?"	<code>grep</code>
3. Inspection	"What does this file actually do?"	<code>cat</code>
4. Context	"What other files call this function?"	<code>grep</code> references

Philosophy: Active exploration over passive consumption. The model **DECIDES** the search strategy, iterating dynamically until it understands the codebase. There's no pre-computed index to consult — it reads the live source of truth every time.

8 Why Agentic Search Works

Three decisive advantages make agentic search the right choice for code:

1. PRECISION — Exact Symbol Matches

Code has exact symbols. `grep` provides exact matches with **zero ambiguity**. Embeddings often return "fuzzy positives" — a function called `authenticate_user()` might semantically match `authorize_request()`, but they do completely different things.

2. FRESHNESS — Always Current

Codebases change every second. Agentic search reads the **actual current file** from disk. There is zero latency waiting for a background indexer to re-chunk, re-embed, and re-sync. The moment you save a file, the agent sees the new version.

3. SECURITY & SIMPLICITY

No external index to build, maintain, or secure. No sensitive code snippets stored in third-party vector databases. Everything stays local, within the agent's environment.

"Even for our own codebase [at Anthropic], it's very sensitive. We don't want to upload it to a third-party thing." — **Boris Cherny, Anthropic**

Result: Semantic search accuracy improved by **12.5%** when Cursor invested heavily in making indexing fast, fresh, and secure — but Claude Code sidesteps the entire problem.

9 The Indexed RAG Approach (Cursor)

While Claude Code favors live shell commands, Cursor invests massively in a state-of-the-art **Indexed RAG** architecture. The engineering behind it is remarkable.

The Four Pillars

Pillar	Technology	What It Does
SMART CHUNKING	Tree-sitter (AST parser)	Parses code into Abstract Syntax Trees. Chunks at syntactic boundaries (functions, classes) so logic is never split in half.
INCREMENTAL SYNC	Merkle trees	Detects <i>exactly</i> which files changed. Only re-embeds modified files, keeping the index fresh with minimal compute.
TEAM REUSE	Shared indexes	Clones share ~92% similarity . New team members reuse existing teammate indexes — setup time drops to near zero.
SEMANTIC SEARCH	Embedding-based retrieval	Enables conceptual queries like " <i>Where do we handle rate limiting?</i> " without knowing any specific function or variable name.

[!TIP] **The key advantage of Indexed RAG:** You can ask *conceptual* questions. With grep, if you don't know the function name, you can't find it. With embeddings, semantic similarity bridges that gap.

10 Comparing the Struggles

Neither approach is perfect. Choosing between them depends entirely on your specific workflow.

The Honest Trade-offs

Trade-off	Claude Code (Agentic)	Cursor (Indexed RAG)
 Speed	 Slower — Sequential "20 questions" with the filesystem. Each grep/cat loads results into context.	 Fast — Instant lookup against pre-computed vector index.
 Cost	 Higher — Grep results load directly into the LLM's context, burning tokens.	 Lower — Only the top vector hits enter the prompt.

Trade-off	Claude Code (Agentic)	Cursor (Indexed RAG)
 Freshness	✓ Perfect — Reads live from disk. Always the absolute ground truth.	✗ Delayed — Even a 5-minute sync delay can cause subtle file drift.
 Conceptual Queries	✗ Hard — Can't grep for something if you don't know its name.	✓ Excellent — "Similar ideas" naturally surface in embedding space.
 Noise Profile	✓ Low — Exact symbols only. Zero false positives.	✗ High — "Similar but wrong" results can distract the model.
 Setup	✓ None — Works immediately on any codebase.	✗ Hours — Large repositories can take significant time to index initially.

Neither is perfect; they offer different trade-offs for different workflows.

11 The 2026 Consensus & Summary

The Best Teams Use Both

Tool	Best For
Claude Code (Agentic)	Deep, autonomous, complex multi-file refactors where absolute code precision is required.
Cursor (Indexed RAG)	Quick, visual, interactive work. Semantic discovery and rapid UI iteration.

The Core Lesson

How the model relates to context is the fundamental design decision of modern AI systems.

Whether it's:

- **RLM** — Context as a programmatic environment explored with code
- **Claude Code** — Context as a live filesystem explored with shell tools
- **Cursor** — Context as a semantic index explored with vector search

These are all just different architectural answers to the same core bottleneck: *the model can't read everything, so what should it read, and how should it choose?*

The Paradigm Shift

2023: "Give the model MORE context"	→ Bigger windows, stuff everything in
2024: "Give the model BETTER context"	→ RAG, chunking, reranking
2026: "Let the model CHOOSE its own context"	→ RLM, Agentic Search, Active Exploration

[!CAUTION] "Understanding why each approach fits its domain is what separates someone who uses AI tools from someone who builds them."

Don't blindly stuff the context window.

Hands-On: Building an RLM from Scratch

The Colab notebook ([class_8.ipynb](#)) builds a complete Recursive Language Model in **7 progressive parts**. By the end, you will have proven — with real code and real outputs — that a model writing code to explore context beats a model drowning in context.

Part 0: Setup

```
import requests, json, re, io, sys, traceback
from contextlib import redirect_stdout, redirect_stderr
from google.colab import userdata

OPENROUTER_API_KEY = userdata.get('OPENROUTER_API_KEY')
ROOT_MODEL = "google/gemini-3-flash-preview" # Root agent (stronger)
SUB_MODEL = "google/gemini-3-flash-preview" # Sub-agent (can be cheaper)

def llm_call(prompt, system="", model=ROOT_MODEL, max_tokens=4000):
    """Call an LLM via OpenRouter. Returns the text response."""
    msgs = []
    if system:
        msgs.append({"role": "system", "content": system})
    msgs.append({"role": "user", "content": prompt})
    r = requests.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={"Authorization": f"Bearer {OPENROUTER_API_KEY}"},
        json={"model": model, "messages": msgs, "max_tokens": max_tokens}
    )
    return r.json()["choices"][0]["message"]["content"]
```

Part 1: The Problem — Long Context Fails

Task: Count how many engineers are based in Tokyo in a synthetic dataset.

```
entries = generate_dataset(5000) # 5,000 people with names, cities, professions
dataset_text = "\n".join(entries) # 265,745 characters

ground_truth = sum(1 for e in entries if "engineer" in e and "Tokyo" in e)
# 🔄 Ground truth: 40 engineers in Tokyo
```

Vanilla Approach — Stuff Everything Into the Prompt:

```

vanilla_prompt = f"""Here is a dataset of people. Count EXACTLY how many are
engineers in Tokyo.
Return ONLY the number, nothing else.

{dataset_text}"""

# Prompt length: 265,860 characters
vanilla_answer = llm_call(vanilla_prompt)

```

Result:

```

Vanilla LLM answer: 12
Ground truth:      40
Correct?           ✗ No

```

[!WARNING] **The model failed on a trivial counting task.** Not because it's "dumb," but because 265K characters of near-identical entries ("Entry 1: Uma D., age 37, chef... Entry 4999: ...") overwhelm the attention mechanism. This is context rot in action — provably, undeniably.

🌀 Part 2: Build the REPL Environment

The REPL is where the model's code runs. The context lives here — **NOT in the LLM's prompt.**

```

class RLMRepl:
    """A REPL environment for RLM.

    The context is stored as a Python variable.
    The model writes code that runs here.
    print() output is captured and returned to the model.
    """
    def __init__(self, context: str, max_output_chars: int = 5000):
        self.final_answer = None
        self.sub_call_count = 0

        # The namespace where the model's code runs
        self.namespace = {
            "context": context,          # <-- the long input lives HERE, not in
prompt                                     completion
            "FINAL": self._final,        # call FINAL("answer") to signal
            "llm_query": self._llm_query, # recursive sub-LLM call
            "re": re, "json": json,      # standard library modules
            "len": len, "sum": sum, "print": print,
            # ... (sorted, min, max, range, enumerate, etc.)
        }

    def _final(self, answer):

```

```

        """Called by the model to submit its final answer."""
        self.final_answer = str(answer)
        print(f"[FINAL ANSWER SUBMITTED: {answer}]")

def _llm_query(self, query: str, sub_context: str = "") -> str:
    """Recursive sub-LLM call.

    The result comes back as a STRING variable –
    NOT loaded into the parent's context window.
    """
    self.sub_call_count += 1
    prompt = f"{query}\n\nContext:\n{sub_context}" if sub_context else query
    return llm_call(prompt, model=SUB_MODEL, max_tokens=2000)

def execute(self, code: str) -> str:
    """Execute Python code in the REPL. Returns captured stdout."""
    stdout_capture = io.StringIO()
    try:
        with redirect_stdout(stdout_capture):
            exec(code, self.namespace)
        output = stdout_capture.getvalue()
    except Exception as e:
        output = f"ERROR: {type(e).__name__}: {e}"
    # Truncate if too long – model shouldn't be overwhelmed
    if len(output) > self.max_output_chars:
        output = output[:self.max_output_chars] + f"\n... [truncated]"
    return output

```

[!NOTE] **Critical design:** The `llm_query()` function returns its result as a Python *string variable*. The sub-LLM's full prompt and response never enter the Root LLM's context — preventing context growth from sub-agent calls.

🌀 Part 3: The RLM Agent Loop

The system prompt is the architecture — it tells the model HOW to use the REPL:

```
RLM_SYSTEM_PROMPT = """You are an RLM (Recursive Language Model) agent.
```

```

You have access to a Python REPL environment. The user's data is stored
in a variable called `context` – it may be very long (millions of characters).
You CANNOT see the context directly. You must write Python code to explore it.

```

Available tools:

- `context` – the full input text (Python string variable)
- `print()` – use this to see output from your code
- `\_llm\_query(query, sub\_context)` – call a sub-LLM to analyze a chunk.
The sub-LLM's response is returned as a string. It does NOT enter your context.
- `FINAL(answer)` – call this when you have the final answer.

Strategy:

1. First, check the size: ``print(len(context))``
2. Peek at the structure: ``print(context[:500])``
3. Use code to search, filter, count, or slice the data
4. For complex subtasks, use ``llm_query()`` to delegate to a sub-LLM
5. When done, call ``FINAL(your_answer)``

Rules:

- Write ONLY Python code. No markdown, no explanation.
- Your code block must be wrapped in ````python ... ````
- Use `print()` to see results – you only see what you print.
- Variables persist between steps (like Jupyter cells).

The Loop itself (~60 lines) follows the exact same ReAct pattern from Week 08:

```
def run_rlm(query: str, context: str, max_iterations: int = 10):
    repl = RLMRepl(context)
    history = []

    # First message: query + context SIZE (but NOT the context itself!)
    user_msg = f"Task: {query}\nThe data is in `context` ({len(context)} characters long)."
    history.append({"role": "user", "content": user_msg})

    for i in range(max_iterations):
        # 1. Ask the LLM to write code
        response = llm_call_with_history(history)
        history.append({"role": "assistant", "content": response})

        # 2. Extract and execute the code in the REPL
        code = extract_code(response)
        output = repl.execute(code)

        # 3. Check if FINAL() was called
        if repl.final_answer is not None:
            return {"answer": repl.final_answer, "iterations": i + 1,
                    "sub_calls": repl.sub_call_count}

        # 4. Feed REPL output back to the LLM for next iteration
        history.append({"role": "user",
                        "content": f"REPL\noutput:\n```\n{output}\n```\nContinue."})

    return {"answer": None, "iterations": max_iterations}
```

Part 4: Run the RLM — Same Task, Different Result

Same dataset. Same question. But now the context never enters the prompt.

```
=====
ITERATION 1/8
=====

📄 LLM wrote:

print(f"Context length: {len(context)}")
print(context[:1000])

📄 REPL output:

Context length: 265745
Entry 1: Uma D., age 37, chef, based in New York
Entry 2: Hank E., age 65, doctor, based in Bangkok
...
```

```
=====
ITERATION 2/8
=====

📄 LLM wrote:

import re
pattern = r", engineer, based in Tokyo"
matches = re.findall(pattern, context, re.IGNORECASE)
count = len(matches)
print(f"Found {count} engineers based in Tokyo.")
FINAL(count)

📄 REPL output:

Found 40 engineers based in Tokyo.
[FINAL ANSWER SUBMITTED: 40]

📄 FINAL ANSWER: 40
Iterations: 2
Sub-LLM calls: 0
```

The Head-to-Head Comparison:

	Vanilla LLM	RLM
Answer	12 ❌	40 ✅
Ground Truth	40	40
Tokens Through Attention	~70,000+	~500
Iterations	1 (one blind shot)	2 (peek + solve)
How It Solved	Tried to count by reading everything	Wrote 3 lines of regex

The model never processed 5,000 entries through its attention mechanism. It wrote a regex that processed them in milliseconds.

🔗 Part 5: A Harder Task — Where Sub-LLMs Shine

Counting is pure code — no LLM understanding needed. But what about tasks requiring *comprehension*, like sentiment analysis? This is where `llm_query()` (recursive sub-LLM calls) becomes essential.

Task: Analyze 200 restaurant reviews. For each restaurant, determine review count, overall sentiment, and most-mentioned dish.

What the RLM Did (autonomously):

- 1. **Peeked** at the data to discover the format (`Review N [Restaurant]: text`)
- 2. **Wrote regex** to group reviews by restaurant name (code handles parsing)
- 3. **Called `llm_query()`** for each restaurant's reviews — 10 sub-LLM calls, one per restaurant (LLM handles understanding)
- 4. **Compiled** the results and called `FINAL()`

🔍 FINAL ANSWER:

- Spice Route:	23 reviews	Sentiment: mixed	Top dish: burger
- Ocean Plate:	29 reviews	Sentiment: positive	Top dish: sushi
- Chez Marie:	16 reviews	Sentiment: mixed	Top dish: salad
- The Garden Kitchen:	16 reviews	Sentiment: mixed	Top dish: curry
- Green Bowl:	21 reviews	Sentiment: mixed	Top dish: curry
- Fire & Stone:	19 reviews	Sentiment: mixed	Top dish: pizza
- Bella Vita:	18 reviews	Sentiment: mixed	Top dish: steak
- The Grill Room:	19 reviews	Sentiment: mixed	Top dish: sushi
- Noodle Bar:	19 reviews	Sentiment: mixed	Top dish: tacos
- Sakura House:	20 reviews	Sentiment: mixed	Top dish: risotto

Iterations: 2 | Sub-LLM calls: 10

[!IMPORTANT] The Key RLM Pattern:

- **Code** handles what code is good at: parsing, grouping, counting, filtering
- **Sub-LLMs** handle what LLMs are good at: understanding, summarizing, analyzing sentiment
- **The Root LLM** orchestrates the whole thing — like a project manager delegating to specialists

Each sub-LLM only sees ONE restaurant's reviews — small, clean context. **No context rot.**

📊 Part 7: Scale Test — Does RLM Actually Scale?

20,000 entries. 1,076,295 characters. Ground truth: 146 engineers in Tokyo.

=====

SCALE TEST RESULTS

```
=====
Dataset size: 20,000 entries (1,076,295 characters)
RLM answer: 146
Ground truth: 146
Correct? ☒
Iterations: 2
Sub-LLM calls: 0
```

The RLM processes 20,000 entries without any of them entering the LLM's context window. The model wrote a few lines of Python that ran in milliseconds.

A vanilla LLM would need to process ~1M characters through attention. The RLM processed ~500 tokens total. **Same accuracy, 2000x fewer tokens.**

Part 8: Recap — What We Built

Traditional LLM:

prompt (with ALL the data) → model → answer

Problem: Context rot. Limited by window size. Accuracy degrades.

Our RLM:

query (small) → Root LLM → writes code → REPL executes it



- ☒ context stored as variable in REPL (never in prompt)
- ☒ llm_query() for recursive sub-LLM calls (never in parent context)
- ☒ FINAL() to submit the answer
- ☒ Scales to 10M+ tokens with zero architectural changes

Excalidraw Masterclass

The whiteboarding sessions covered advanced architectural concepts that tie the entire course arc together — from retrieval to agents to the transformer itself.

1. Anthropic's Contextual Retrieval (The Pre-Processing Fix)

Chunking a document naturally **destroys the context** of individual paragraphs. A chunk saying "*The company's revenue grew by 2%*" leaves the vector database unable to determine *which* company or *what* year.

The Fix: Before embedding, use an LLM to prepend overarching document context to *every single chunk*.

Example Chunk

BEFORE (Lossy)

"The company's revenue grew by 2%."

Example Chunk

AFTER (Contextualized)	"[From ACME Corp Q2 2023 SEC Filing] The company's revenue grew by 2%."
---------------------------	---

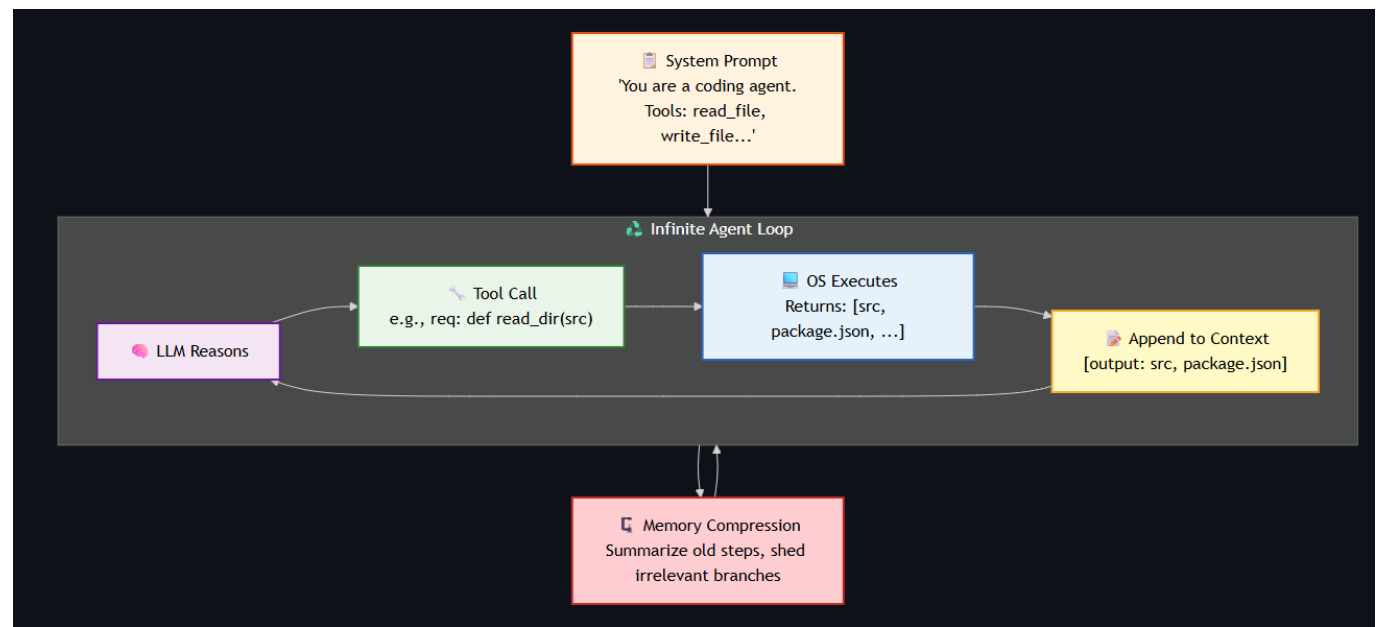
This makes the embedding capture *what the chunk is about*, not just what it literally says. Combined with BM25 and reranking, Anthropic reported **67% fewer retrieval failures** — a one-time ingestion cost for a permanent query-time gain.

2. RAG Architecture: The Three Pillars (Revised)

Pillar	Steps	Key Processes
Ingestion	Parsing → Chunking → Contextualization → Embedding	Converts raw documents into searchable vectors
Retrieval	Query Transform → Hybrid Search → Reranking	Casts wide net for recall, then filters for precision
Generation	Context Assembly → Prompt → LLM → Cited Answer	Augments the prompt and generates a grounded, verifiable response

3. Anatomy of a Coding Agent (Under the Hood)

Building a CLI-native tool like Claude Code requires wrapping the Transformer in an infinite REPL loop interacting directly with the Operating System:



Why Memory Compression Matters: Because the agent loop is technically infinite and multi-turn, the agent must constantly compress its memory — summarizing old investigation steps and shedding irrelevant context branches — so it doesn't blow past its token limit during extended research tasks.

4. The Transformer Foundation: Causal Masking

Underneath all of this agentic orchestration is the fundamental engine of **Next-Token Prediction** via causal masking — the same mechanism from Week 07's **MiniLLM**:

```
# The lower-triangular mask that makes autoregressive generation possible
mask = torch.triu(torch.ones(seq, seq, device=x.device), diagonal=1).bool()
```

Concept

Input	'they enj'
Target	'hey enjo' (shifted right by one position)
Mask	Upper-triangular matrix of True values → attention scores set to -inf
Effect	When predicting o from enj, the model can only see past tokens, never future ones

This mechanism is what generates both conversational answers *and* the structured JSON tool calls that make agentic behavior possible. Every ACTION: grep, every FINAL(42), every llm_query() — it all comes from next-token prediction under a causal mask.

 Further Reading & Resources

Resource	Link
RLM Paper	arxiv.org/abs/2512.24601 — Zhang, Kraska & Khattab (MIT CSAIL)
RLM Official Code	github.com/alexzhang13/rlm
Prime Intellect Blog	primeintellect.ai/blog/rlm
Alex Zhang's Blog	alexzhang13.github.io/blog/2025/rlm/
Context Rot Research	research.trychroma.com/context-rot — Chroma (2025)
ChatGPT Memory Reverse-Engineering	manthanguptaa.in/posts/chatgpt_memory/
Anthropic Contextual Retrieval	anthropic.com/news/contextual-retrieval

Made with  by Akshat Shethia